

# หน่วยที่ 4

## คำสั่งการตรวจสอบและควบคุม

1. อธิบายหลักการทำงานของคำสั่ง if, if-else
2. เขียนโปรแกรมด้วยคำสั่ง if-else if ตรวจสอบหลายเงื่อนไข
3. อธิบายหลักการทำงานของคำสั่ง switch-case
4. เขียนโปรแกรมด้วยคำสั่ง switch-case
5. อธิบายหลักการทำงานของคำสั่ง Loop
6. เขียนโปรแกรมด้วยคำสั่ง Loop
7. อธิบายหลักการทำงานของคำสั่ง Array
8. เขียนโปรแกรมด้วยคำสั่ง Array
9. ประยุกต์ใช้ตัวดำเนินการเชิงตรรกะร่วมกับเงื่อนไข





หน่วยที่ 4 คำสั่งการตรวจสอบและควบคุม

วัตถุประสงค์เชิงพฤติกรรม 6 ข้อ

- 1. อธิบายหลักการทำงานของคำสั่ง if, if-else
- 2. เขียนโปรแกรมด้วยคำสั่ง if-else if ตรวจสอบหลายเงื่อนไข
- 3. อธิบายหลักการทำงานของคำสั่ง switch-case
- 4. เขียนโปรแกรมด้วยคำสั่ง switch-case
- 5. อธิบายหลักการทำงานของคำสั่ง Loop
- 6. เขียนโปรแกรมด้วยคำสั่ง Loop
- 7. อธิบายหลักการทำงานของคำสั่ง Array
- 8. เขียนโปรแกรมด้วยคำสั่ง Array
- 9. ประยุกต์ใช้ตัวดำเนินการเชิงตรรกะร่วมกับเงื่อนไข

4.1 หลักการของคำสั่งตรวจสอบเงื่อนไข (ฉบับละเอียด)

4.1.1 จากโปรแกรมแบบลำดับสู่โปรแกรมแบบเลือก

จากหน่วยที่ 3 ผู้เรียนได้ศึกษาการเขียนโปรแกรมแบบลำดับ ซึ่งทุกคำสั่งทำงานเรียงต่อกันไปตามลำดับก่อนหลังโดยไม่มีการแตกกิ่งก้านหรือแยกทางเลือกใด ๆ เปรียบเสมือนการเดินทางบนเส้นทางตรงเส้นเดียวตั้งแต่ต้นจนจบ แต่ในสถานการณ์จริง โปรแกรมส่วนใหญ่จำเป็นต้อง "ตัดสินใจ" เลือกทำงานแตกต่างกันไปตามสถานการณ์ที่พบ เช่น ถ้าคะแนนสอบผ่านเกณฑ์ให้แสดงว่า "ผ่าน" แต่ถ้าไม่ผ่านเกณฑ์ให้แสดงว่า "ไม่ผ่าน" ลักษณะการทำงานเช่นนี้เรียกว่า การเขียนโปรแกรมแบบเลือก (Selection) ซึ่งอาศัย คำสั่งตรวจสอบเงื่อนไข (Conditional Statement) เป็นกลไกหลักในการควบคุมทิศทางการทำงานของโปรแกรม

4.1.2 หลักการทำงานของคำสั่งตรวจสอบเงื่อนไข

คำสั่งตรวจสอบเงื่อนไขจะทำการประเมินค่านิพจน์ทางตรรกะ (Logical Expression) ซึ่งผลลัพธ์ที่ได้จะมีเพียง 2 ค่าเท่านั้น คือ

- true (จริง) — โปรแกรมจะเข้าทำงานตามคำสั่งที่กำหนดไว้สำหรับกรณีเงื่อนไขเป็นจริง
- false (เท็จ) — โปรแกรมจะข้ามคำสั่งนั้นไป หรือไปทำงานตามคำสั่งอื่นที่กำหนดไว้สำหรับกรณีเป็นเท็จแทน

ตัวอย่างนิพจน์ทางตรรกะที่ใช้เป็นเงื่อนไขได้ เช่น score >= 50, age < 18, grade == "A" ซึ่งล้วนใช้ตัวดำเนินการเปรียบเทียบที่ได้ศึกษามาแล้วในหน่วยที่ 3

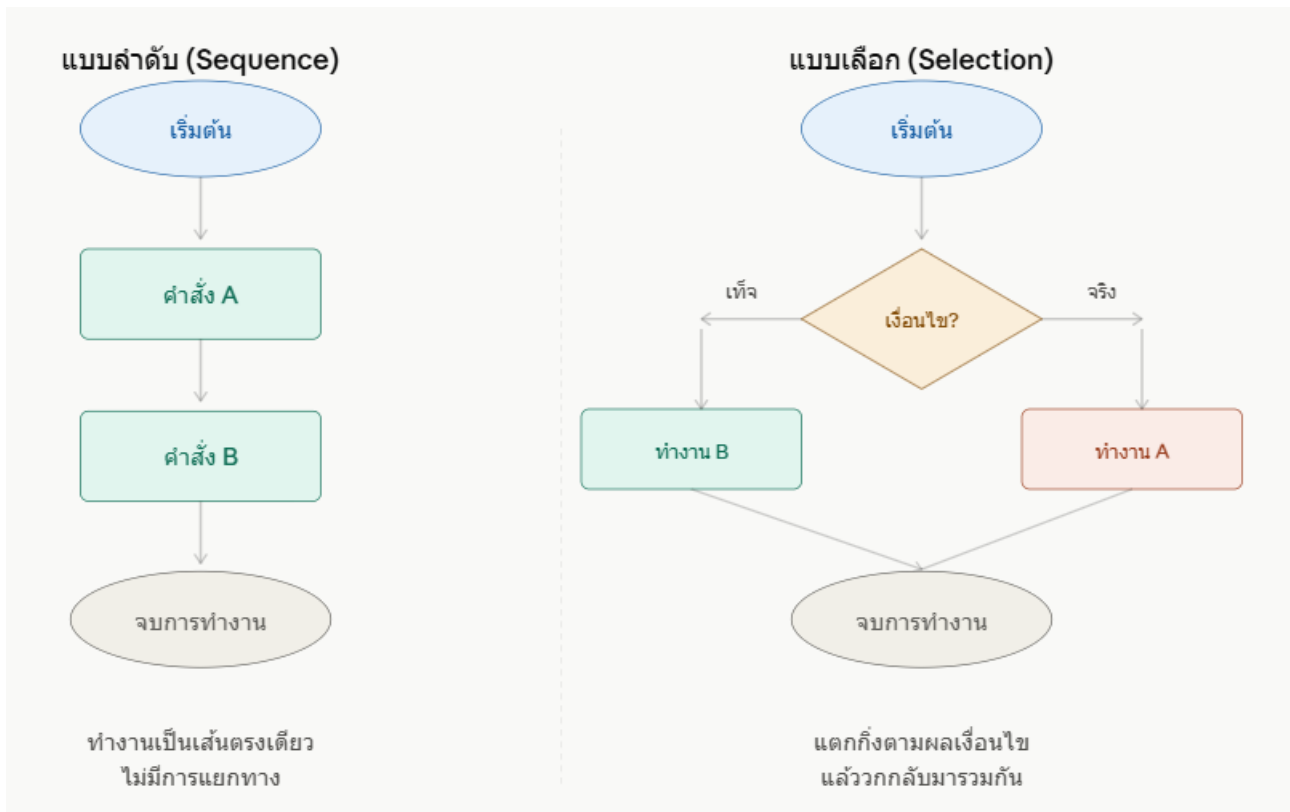
4.1.3 สัญลักษณ์ผังงานรูปข้าวหลามตัด (Decision Symbol)

ในการเขียนผังงานแบบลำดับที่ผ่านมา ผู้เรียนได้รู้จักสัญลักษณ์พื้นฐาน 4 แบบ คือ วงรี สี่เหลี่ยมผืนผ้า สี่เหลี่ยมด้านขนาน และ ลูกศร แต่เมื่อโปรแกรมมีการตัดสินใจเลือกทางเดิน จะต้องเพิ่มสัญลักษณ์ใหม่เข้ามา นั่นคือ สี่เหลี่ยมข้าวหลามตัด (Decision Symbol)

| สัญลักษณ์             | รูปร่าง | ความหมาย           | ลักษณะพิเศษ                                                 |
|-----------------------|---------|--------------------|-------------------------------------------------------------|
| สี่เหลี่ยมข้าวหลามตัด | ◇       | จุดตรวจสอบเงื่อนไข | มีทางออก 2 ทาง เสมอ คือ จริง (Yes/True) และ เท็จ (No/False) |

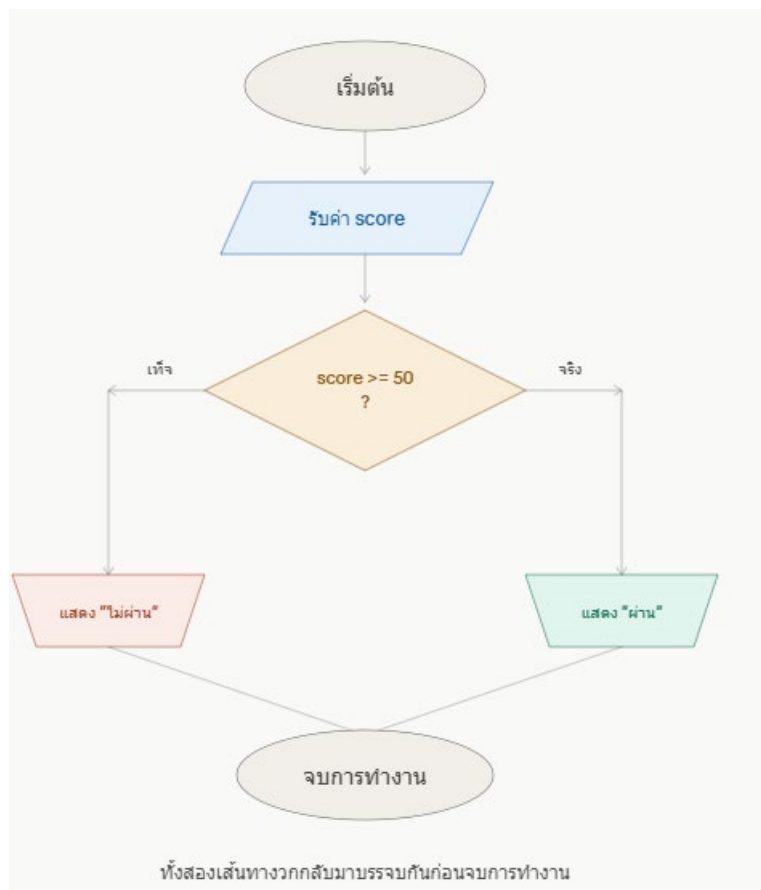
ข้อแตกต่างสำคัญจากสัญลักษณ์อื่น: สัญลักษณ์ทุกแบบที่ผ่านมามีทางเข้าและทางออกอย่างละ 1 ทางเท่านั้น แต่สัญลักษณ์ข้าวหลามตัดมีทางออก 2 ทาง เสมอ ทำให้ผังงานเริ่มมีลักษณะการแตกกิ่งก้าน (Branching) ซึ่งเป็นจุดเปลี่ยนสำคัญจากผังงานแบบเส้นตรงในหน่วยที่ 3

ต่อไปนี้เป็นแผนภาพเปรียบเทียบโครงสร้างผังงานแบบลำดับ (เส้นตรง) กับผังงานแบบเลือก (แตกกิ่ง) เพื่อให้เห็นความแตกต่างชัดเจน



#### 4.1.4 ตัวอย่างผังงานการตรวจสอบเงื่อนไขในสถานการณ์จริง

เพื่อให้เห็นภาพการใช้งานสัญลักษณ์ข้างหลามตัดในบริบทจริง ขอยกตัวอย่างผังงานสำหรับโปรแกรมตรวจสอบว่านักเรียนสอบผ่านหรือไม่ผ่าน



#### 4.1.5 โค้ด C# ที่สอดคล้องกับผังงานข้างต้น

```
using System;
class Program
{
    static void Main()
    {
        Console.Write("กรอกคะแนนสอบ: ");
        int score = int.Parse(Console.ReadLine());
        if (score >= 50)
        {
            Console.WriteLine("ผ่าน");
        }
        else
        {
            Console.WriteLine("ไม่ผ่าน");
        }
    }
}
```

ตารางเปรียบเทียบผังงานกับโค้ด

| ขั้นตอนในผังงาน                       | คำสั่งในโค้ด C#                            |
|---------------------------------------|--------------------------------------------|
| สี่เหลี่ยมด้านขนาน "รับค่า score"     | int score = int.Parse(Console.ReadLine()); |
| สี่เหลี่ยมข้าวหลามตัด "score >= 50 ?" | if (score >= 50)                           |
| เส้นทาง "จริง" → แสดง "ผ่าน"          | { Console.WriteLine("ผ่าน"); }             |
| เส้นทาง "เท็จ" → แสดง "ไม่ผ่าน"       | else { Console.WriteLine("ไม่ผ่าน"); }     |

#### 4.1.6 ข้อสังเกตสำคัญเกี่ยวกับผังงานแบบเลือก

1. ทางออกทั้งสองทางของสัญลักษณ์ข้าวหลามตัดต้องมีป้ายกำกับเสมอ โดยทั่วไปกำกับด้วยคำว่า "จริง/เท็จ" หรือ "Yes/No" เพื่อไม่ให้เกิดความสับสนว่าทางใดคือทางที่เงื่อนไขเป็นจริง
2. เส้นทางทั้งสองสามารถวกกลับมาบรรจบกันได้ ก่อนจบการทำงาน ดังตัวอย่างข้างต้น หรืออาจแยกไปทำงานต่อคนละทิศทางเลยก็ได้ ขึ้นอยู่กับการออกแบบโปรแกรม
3. เงื่อนไขต้องเขียนให้ชัดเจนและวัดผลได้จริง เช่น `score >= 50` ไม่ใช่ข้อความกำกวมอย่าง "คะแนนดี" ซึ่งคอมพิวเตอร์ไม่สามารถประเมินค่าได้

### 4.3 คำสั่ง if-else if (การตรวจสอบหลายเงื่อนไข)

#### 4.3.1 ทำไมต้องมี if-else if

จากหัวข้อ 4.2 ผู้เรียนได้ทราบแล้วว่า if-else เหมาะสำหรับสถานการณ์ที่มีเพียงสองทางเลือก เช่น ผ่าน/ไม่ผ่าน แต่ในชีวิตจริงมีสถานการณ์อีกมากที่มีทางเลือกมากกว่า 2 ทาง เช่น การตัดเกรดที่มีถึง 5 ระดับ (A, B, C, D, F) หากใช้ if-else เพียงอย่างเดียวจะไม่สามารถรองรับได้ ภาษา C# จึงมีโครงสร้าง if-else if เพื่อรองรับการตรวจสอบหลายเงื่อนไขต่อเนื่องกัน

#### 4.3.2 หลักการทำงาน

โปรแกรมจะตรวจสอบเงื่อนไขจากบนลงล่างทีละเงื่อนไข เมื่อพบเงื่อนไขใดเป็นจริงเป็นเงื่อนไขแรก จะเข้าทำงานตามคำสั่งในบล็อกนั้นทันที แล้วออกจากโครงสร้างทั้งหมด โดยไม่ตรวจสอบเงื่อนไขที่เหลืออยู่ด้านล่างอีกต่อไป แม้เงื่อนไขเหล่านั้นจะเป็นจริงด้วยก็ตาม

## รูปแบบคำสั่ง

if (เงื่อนไขที่ 1)

{

คำสั่งที่ 1

}

else if (เงื่อนไขที่ 2)

{

คำสั่งที่ 2

}

else if (เงื่อนไขที่ 3)

{

คำสั่งที่ 3

}

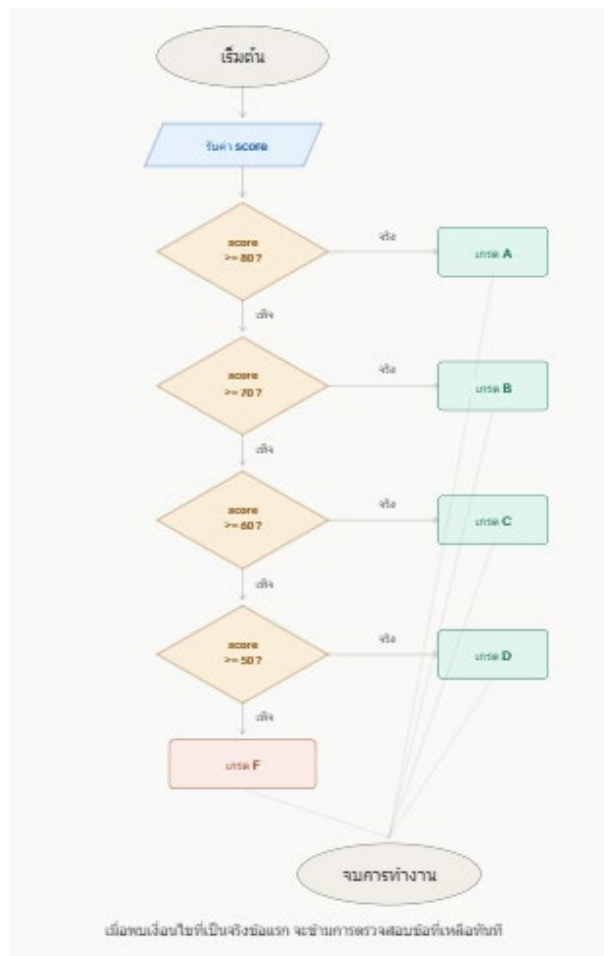
else

{

คำสั่งเมื่อไม่ตรงเงื่อนไขใดเลย

}

ต่อไปนี้เป็นผังงานแสดงการทำงานของ if-else if ในการตัดเกรดตามระดับคะแนน ซึ่งแสดงให้เห็นการตรวจสอบเงื่อนไขต่อเนื่องกันหลายจุด



### 4.3.3 โค้ด C# ที่สอดคล้องกับผังงาน

```
using System;
class Program
{
    static void Main()
    {
        Console.Write("กรอกคะแนนสอบ (0-100): ");
        int score = int.Parse(Console.ReadLine());
        char grade;
        if (score >= 80)
        {
            grade = 'A';
        }
        else if (score >= 70)
        {
            grade = 'B';
        }
        else if (score >= 60)
        {
            grade = 'C';
        }
        else if (score >= 50)
        {
            grade = 'D';
        }
        else
        {
            grade = 'F';
        }
        Console.WriteLine("เกรดที่ได้รับคือ: " + grade);
    }
}
```

#### ตารางแสดงการไล่ตรวจสอบทีละเงื่อนไข (Trace Table)

สมมติผู้ใช้กรอกคะแนน 75 ตารางนี้แสดงการทำงานของโปรแกรมทีละขั้นตอน

| ลำดับ | เงื่อนไขที่ตรวจสอบ     | ผลลัพธ์         | การทำงานของโปรแกรม                              |
|-------|------------------------|-----------------|-------------------------------------------------|
| 1     | score >= 80 → 75 >= 80 | เท็จ            | ข้ามไปตรวจสอบเงื่อนไขถัดไป                      |
| 2     | score >= 70 → 75 >= 70 | จริง            | เข้าทำงาน grade = 'B'; แล้วออกจากโครงสร้างทันที |
| 3     | score >= 60            | (ไม่ถูกตรวจสอบ) | โปรแกรมไม่ตรวจสอบเงื่อนไขนี้อีกต่อไป            |
| 4     | score >= 50            | (ไม่ถูกตรวจสอบ) | โปรแกรมไม่ตรวจสอบเงื่อนไขนี้อีกต่อไป            |

## ผลลัพธ์การรัน

กรอกคะแนนสอบ (0-100): 75

เกรดที่ได้รับคือ: B

### 4.3.4 ข้อผิดพลาดสำคัญ: การเรียงลำดับเงื่อนไขผิด

การเรียงลำดับเงื่อนไขใน if-else if มีความสำคัญอย่างยิ่ง เพราะโปรแกรมจะหยุดที่เงื่อนไขแรกที่เป็นจริงเสมอ ลองพิจารณาตัวอย่างโค้ดที่เขียนผิดต่อไปนี้

// ❌ ตัวอย่างที่เขียนผิด: เรียงเงื่อนไขจากน้อยไปมาก

```
if (score >= 50)
{
    grade = 'D';
}
else if (score >= 60)
{
    grade = 'C';
}
else if (score >= 70)
{
    grade = 'B';
}
else if (score >= 80)
{
    grade = 'A';
}
```

เปรียบเทียบผลลัพธ์เมื่อกรอกคะแนน 90

| การเรียงลำดับ                  | ผลลัพธ์ที่ได้ | ถูกต้องหรือไม่ |
|--------------------------------|---------------|----------------|
| เรียงจากมากไปน้อย (>= 80 ก่อน) | เกรด A        | ✅ ถูกต้อง      |
| เรียงจากน้อยไปมาก (>= 50 ก่อน) | เกรด D        | ❌ ผิดพลาด!     |

**คำอธิบาย:** เมื่อกรอกคะแนน 90 หากเรียงเงื่อนไขจากน้อยไปมาก โปรแกรมจะตรวจสอบ score >= 50 ก่อน ซึ่ง 90 >= 50 เป็นจริงทันที จึงกำหนด grade = 'D' และออกจากโครงสร้างทันที โดยไม่ได้ตรวจสอบเงื่อนไข score >= 80 ที่ถูกต้องกว่าเลย ดังนั้นหลักการสำคัญคือต้องเรียงเงื่อนไขจากค่ามากไปน้อยเสมอ เมื่อใช้ตัวดำเนินการ >=

### 4.3.5 ตัวอย่างเพิ่มเติม: โปรแกรมจำแนกประเภทดัชนีมวลกาย (BMI)

เพื่อให้เห็นการประยุกต์ใช้ if-else if ในบริบทอื่น ขอยกตัวอย่างโปรแกรมจำแนกระดับ BMI

```

using System;
class Program
{
    static void Main()
    {
        Console.Write("กรอกค่าดัชนีมวลกาย (BMI): ");
        double bmi = double.Parse(Console.ReadLine());
        string category;

        if (bmi < 18.5)
        {
            category = "น้ำหนักต่ำกว่าเกณฑ์";
        }
        else if (bmi < 23.0)
        {
            category = "น้ำหนักปกติ";
        }
        else if (bmi < 25.0)
        {
            category = "น้ำหนักเกิน";
        }
        else
        {
            category = "โรคอ้วน";
        }
        Console.WriteLine("ผลการประเมิน: " + category);
    }
}

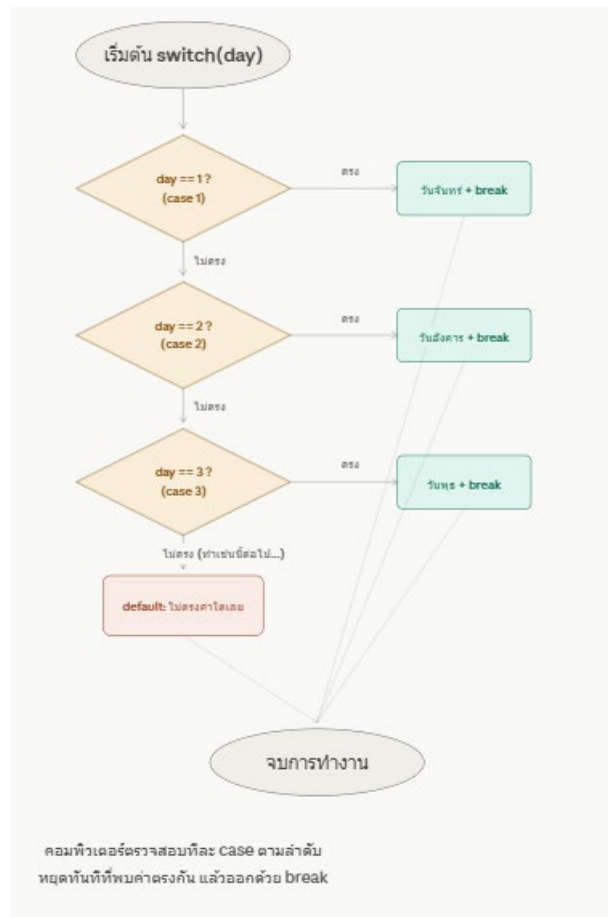
```

**ข้อสังเกต:** ตัวอย่างนี้ใช้ตัวดำเนินการ < (น้อยกว่า) แทน >= จึงต้องเรียงเงื่อนไขจากค่าน้อยไปมาก ซึ่งตรงข้ามกับตัวอย่างเกรดคะแนน แสดงให้เห็นว่าการเรียงลำดับเงื่อนไขขึ้นอยู่กับทิศทางของตัวดำเนินการเปรียบเทียบที่เลือกใช้เสมอ

#### 4.4 หลักการทำงานของ switch-case แบบที่ละขั้นตอน

ต่างจากที่แผนภาพก่อนหน้านี้แสดงภาพรวมแบบแตกกิ่งพร้อมกันทุก case ในความเป็นจริงคอมพิวเตอร์จะตรวจสอบค่าที่ละ case ตามลำดับจากบนลงล่าง จนกว่าจะพบค่าที่ตรงกัน ดังผังงานต่อไปนี้





### Trace Table: การไล่ตรวจสอบทีละ case

สมมติผู้ใช้กรอกค่า **day = 5** จากโปรแกรมแสดงชื่อวันในสัปดาห์ (หัวข้อ 4.4.4) ตารางนี้แสดงลำดับการทำงานจริงของคอมพิวเตอรืทีละขั้นตอน

| ลำดับ | case ที่ตรวจสอบ            | เปรียบเทียบ     | ผลลัพธ์ | การทำงานของโปรแกรม                                                 |
|-------|----------------------------|-----------------|---------|--------------------------------------------------------------------|
| 1     | case 1:                    | 5 == 1          | เท็จ    | ข้ามไปตรวจสอบ case ถัดไป                                           |
| 2     | case 2:                    | 5 == 2          | เท็จ    | ข้ามไปตรวจสอบ case ถัดไป                                           |
| 3     | case 3:                    | 5 == 3          | เท็จ    | ข้ามไปตรวจสอบ case ถัดไป                                           |
| 4     | case 4:                    | 5 == 4          | เท็จ    | ข้ามไปตรวจสอบ case ถัดไป                                           |
| 5     | case 5:                    | 5 == 5          | จริง    | เข้าทำงาน dayName = "วันศุกร์"; แล้วพบ break; ออกจากโครงสร้างทันที |
| 6     | case 6:, case 7:, default: | (ไม่ถูกตรวจสอบ) | —       | โปรแกรมไม่ตรวจสอบส่วนที่เหลืออีกต่อไป เพราะออกจากโครงสร้างไปแล้ว   |

### ผลลัพธ์การรัน

กรอกตัวเลขวัน (1-7): 5

วันที่ 5 คือ วันศุกร์

### ข้อสรุปหลักการทำงาน 3 ข้อ

1. ตรวจสอบตามลำดับจากบนลงล่างเสมอ — คอมพิวเตอรืไม่ได้ตรวจสอบทุก case พร้อมกัน แต่ไล่ทีละรายการ
2. หยุดทันทีที่พบค่าตรงกัน — เมื่อพบ case ที่ตรงกับค่าของตัวแปร โปรแกรมจะไม่ตรวจสอบ case ที่เหลือต่อ (ต่างจากการวนตรวจสอบทุกเงื่อนไขจนจบ)
3. break คือกลไกหยุดการทำงาน — หากไม่มี break โปรแกรมจะไหลต่อไปทำงานใน case ถัดไปโดยไม่หยุด (ตามที่ได้อธิบายไว้ในหัวข้อ Fall-through ก่อนหน้านี้)

4.5 คำสั่ง switch-case (การตรวจสอบหลายเงื่อนไข)

4.5.1 หลักการและเหตุผลที่ต้องมี switch-case

ในหัวข้อ 4.3 ผู้เรียนได้ศึกษา if-else if สำหรับตรวจสอบหลายเงื่อนไขต่อเนื่องกันแล้ว แต่เมื่อสถานการณ์เป็นการเปรียบเทียบค่าของตัวแปรตัวเดียวกับค่าที่แน่นอนหลายค่า (Equality Comparison) เช่น ตรวจสอบว่าตัวเลข 1-7 หมายถึงวันใดในสัปดาห์ การเขียนด้วย if-else if ซ้อนกันหลายชั้นจะทำให้โค้ดยาวและอ่านยาก ภาษา C# จึงมีคำสั่ง **switch-case** เป็นทางเลือกที่ทำให้โค้ดกระชับและเป็นระเบียบกว่า

4.5.2 หลักการทำงานและรูปแบบคำสั่ง

คำสั่ง switch จะนำค่าของตัวแปรหรือนิพจน์ที่ระบุไปเปรียบเทียบกับค่าคงที่ในแต่ละ case ที่ละรายการ เมื่อพบค่าที่ตรงกัน จะเข้าทำงานในบล็อกนั้นจนกว่าจะพบคำสั่ง break จึงออกจากโครงสร้างทั้งหมด หากไม่มีค่าใดตรงกันเลย จะเข้าทำงานในส่วน default (ถ้ามีการกำหนดไว้)

รูปแบบคำสั่ง

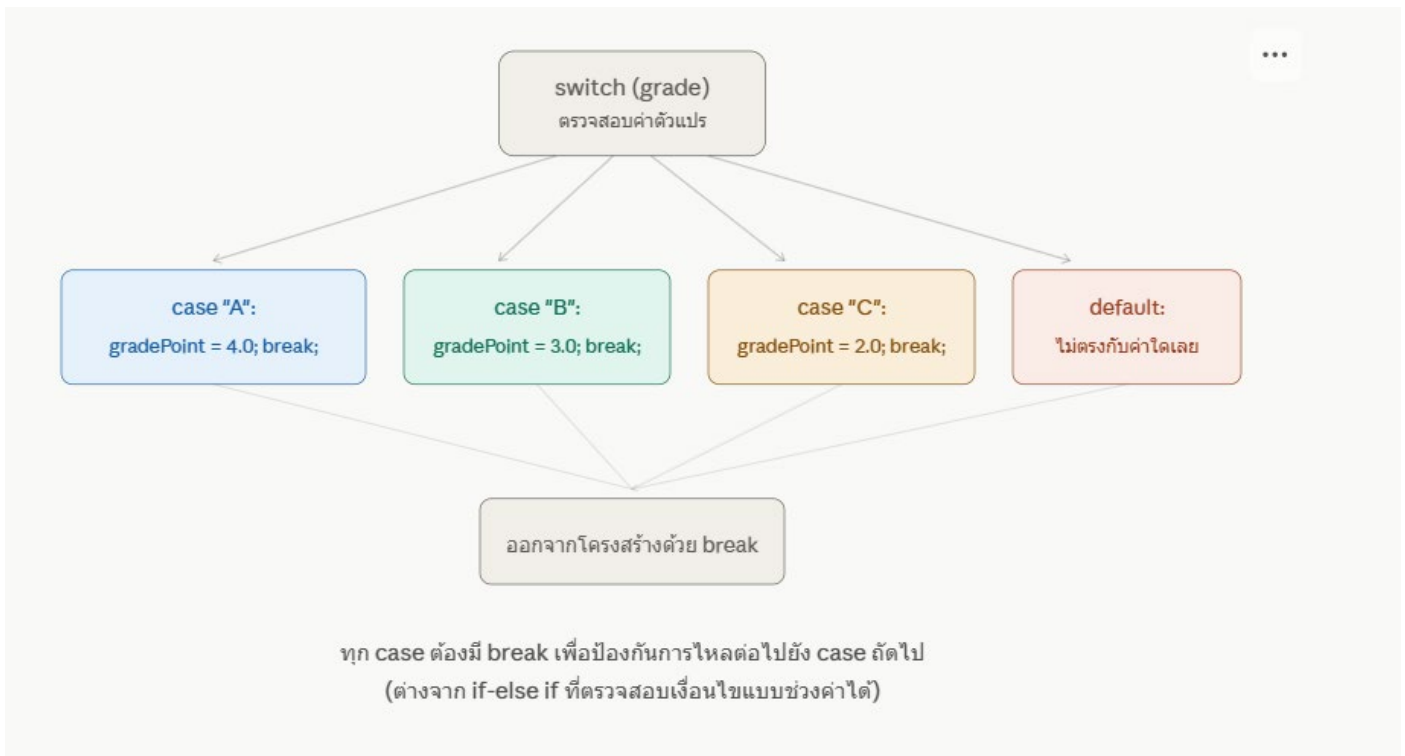
switch (ตัวแปรที่ต้องการตรวจสอบ)

```
{
    case ค่าที่1:
        คำสั่งที่ต้องการให้ทำงาน
        break;
    case ค่าที่2:
        คำสั่งที่ต้องการให้ทำงาน
        break;
    default:
        คำสั่งเมื่อไม่ตรงกับค่าใดเลย
        break;
}
```

คำอธิบายคำสั่งสำคัญ

| คำสั่ง          | ความหมาย                                                     |
|-----------------|--------------------------------------------------------------|
| switch (ตัวแปร) | ระบุตัวแปรหรือนิพจน์ที่ต้องการนำไปเปรียบเทียบ                |
| case ค่า:       | ระบุค่าที่ต้องการเปรียบเทียบ หากตรงกันจะเข้าทำงานในบล็อกนั้น |
| break;          | สั่งให้ออกจากโครงสร้าง switch ทันที (สำคัญมาก ห้ามลืม)       |
| default:        | ทำงานเมื่อค่าที่ตรวจสอบไม่ตรงกับ case ใดเลย (มีหรือไม่ก็ได้) |

ต่อไปนี้เป็นแผนภาพแสดงหลักการทำงานของ switch-case เปรียบเทียบกับ if-else if เพื่อให้เห็นความแตกต่างของโครงสร้าง



#### 4.5.3 ตัวอย่างที่ 1: โปรแกรมแสดงแต้มคะแนนตามระดับคะแนน

```
using System;
class Program
{
    static void Main()
    {
        Console.WriteLine("กรอกเกรดที่ได้รับ (A, B, C, D, F): ");
        string grade = Console.ReadLine();
        double gradePoint;

        switch (grade)
        {
            case "A":
                gradePoint = 4.0;
                break;
            case "B":
                gradePoint = 3.0;
                break;
            case "C":
                gradePoint = 2.0;
                break;
            case "D":
                gradePoint = 1.0;
                break;
            case "F":
```



```

        gradePoint = 0.0;
        break;
    default:
        gradePoint = -1;
        Console.WriteLine("กรอกเกรดไม่ถูกต้อง");
        break;
    }
    if (gradePoint >= 0)
    {
        Console.WriteLine("แต้มคะแนนที่ได้: " + gradePoint);
    }
}
}

```

**ผลลัพธ์ตัวอย่าง (กรอก B)**

กรอกเกรดที่ได้รับ (A, B, C, D, F): B

แต้มคะแนนที่ได้: 3

#### 4.5.4 ตัวอย่างที่ 2: โปรแกรมแสดงชื่อวันในสัปดาห์

```

using System;
class Program
{
    static void Main()
    {
        Console.Write("กรอกตัวเลขวัน (1-7): ");
        int day = int.Parse(Console.ReadLine());
        string dayName;
        switch (day)
        {
            case 1:
                dayName = "วันจันทร์";
                break;
            case 2:
                dayName = "วันอังคาร";
                break;
            case 3:
                dayName = "วันพุธ";
                break;
            case 4:
                dayName = "วันพฤหัสบดี";
                break;
            case 5:
                dayName = "วันศุกร์";
                break;
        }
    }
}

```

```

case 6:
    dayName = "วันเสาร์";
    break;
case 7:
    dayName = "วันอาทิตย์";
    break;
default:
    dayName = "ไม่มีวันนี้ในสัปดาห์";
    break;
}
Console.WriteLine("วันที่ " + day + " คือ " + dayName);
}
}

```

ผลลัพธ์ตัวอย่าง (กรอก 5)

กรอกตัวเลขวัน (1-7): 5

วันที่ 5 คือ วันศุกร์

#### 4.5.5 ข้อผิดพลาดสำคัญ: การลืมเขียน break

หากลืมเขียนคำสั่ง break ในแต่ละ case โปรแกรมจะไหลต่อไปทำงานใน case ถัดไปโดยไม่หยุด (เรียกว่า Fall-through) ซึ่งมักทำให้เกิดผลลัพธ์ที่ไม่ถูกต้อง

// ❌ ตัวอย่างที่เขียนผิด: ลืม break

```

switch (grade)
{
    case "A":
        gradePoint = 4.0;
    case "B":
        gradePoint = 3.0;
        break;
}

```

เปรียบเทียบผลลัพธ์เมื่อกรอกเกรด "A"

| กรณี                            | ผลลัพธ์<br>gradePoint | อธิบาย                                                        |
|---------------------------------|-----------------------|---------------------------------------------------------------|
| มี break ทุก case (ถูกต้อง)     | 4.0                   | ออกจากโครงสร้างทันทีหลังพบ case "A"                           |
| ลืม break ใน case "A" (ผิดพลาด) | 3.0                   | โปรแกรมไหลต่อไปทำงานใน case "B" ทับค่าเดิม ทำให้ได้ผลลัพธ์ผิด |

**ข้อสังเกตสำคัญ:** ภาษา C# บังคับให้ทุก case (ที่มีคำสั่งอยู่ภายใน) ต้องมี break หรือคำสั่งที่ทำให้ออกจากบล็อกเสมอ (เช่น return) มิฉะนั้นตัวแปลภาษาจะแจ้งข้อผิดพลาดขณะคอมไพล์ทันที ซึ่งต่างจากบางภาษา (เช่น C หรือ C++) ที่อนุญาตให้ Fall-through ได้โดยไม่แจ้งเตือน

4.5.6 ตารางเปรียบเทียบ if-else if กับ switch-case (ทบทวน)

| ประเด็นเปรียบเทียบ              | if-else if                                  | switch-case                                                   |
|---------------------------------|---------------------------------------------|---------------------------------------------------------------|
| ลักษณะเงื่อนไข                  | ตรวจสอบได้ทุกรูปแบบ (>, <, >=, ช่วงค่า ฯลฯ) | ตรวจสอบเฉพาะค่าที่ตรงกันแบบเจาะจง (Equality)                  |
| ความอ่านง่ายเมื่อมีหลายเงื่อนไข | อ่านยากขึ้นเมื่อมีเงื่อนไขจำนวนมาก          | อ่านง่ายและเป็นระเบียบกว่า                                    |
| ความเหมาะสม                     | เปรียบเทียบช่วงค่า เช่น คะแนนสอบ, อายุ      | เปรียบเทียบค่าที่แน่นอน เช่น เกรด, ตัวเลือกเมนู, วันในสัปดาห์ |
| คำสั่งสำคัญที่ต้องระวัง         | ลำดับการเรียงเงื่อนไข                       | การใส่ break ในทุก case                                       |

4.6 หลักการทำงานของคำสั่ง Loop (การทำซ้ำ)

1. เหตุผลที่ต้องมีคำสั่ง Loop

จากหน่วยที่ 3 และ 4 ผู้เรียนได้ศึกษาโครงสร้างแบบลำดับ (Sequence) และแบบเลือก (Selection) มาแล้ว แต่ยังมีสถานการณ์อีกมากที่ต้องการให้โปรแกรมทำงานซ้ำ ๆ กันหลายรอบ โดยไม่ต้องเขียนคำสั่งเดิมซ้ำหลายบรรทัด เช่น การพิมพ์ตัวเลข 1 ถึง 100 หากไม่มี Loop ผู้เขียนโปรแกรมจะต้องเขียนคำสั่ง Console.WriteLine() ถึง 100 บรรทัด แต่ด้วยคำสั่ง Loop สามารถเขียนเพียงไม่กี่บรรทัดก็ทำงานซ้ำได้ตามจำนวนที่ต้องการ

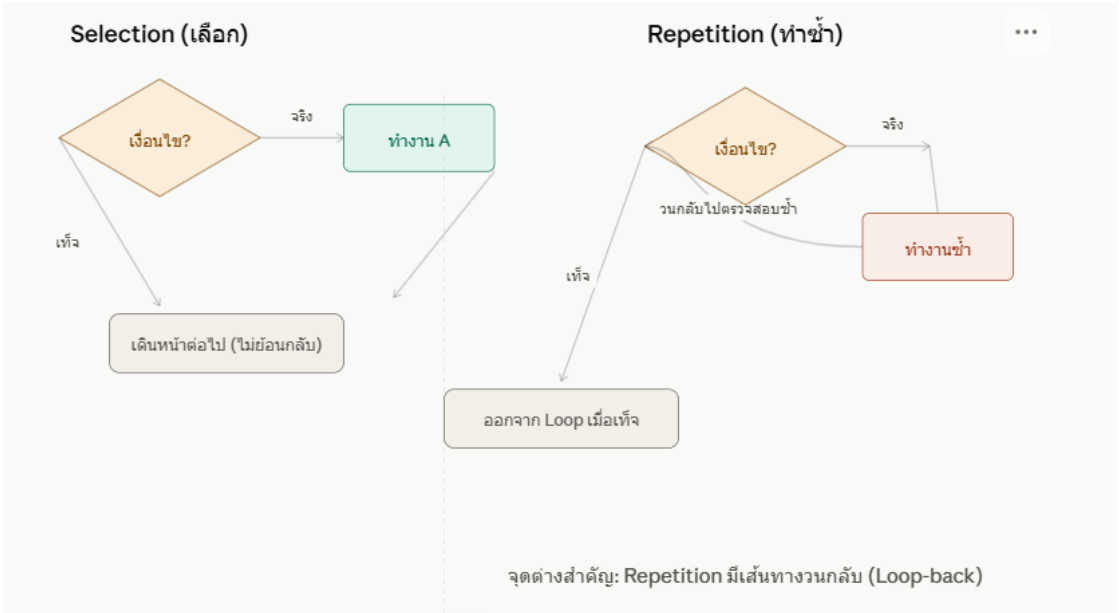
โครงสร้างควบคุมทั้ง 3 รูปแบบที่ผู้เรียนได้ศึกษาครบถ้วนแล้วในภาพรวม

| รูปแบบ             | ลักษณะการทำงาน                          | หน่วยที่ศึกษา |
|--------------------|-----------------------------------------|---------------|
| Sequence (ลำดับ)   | ทำงานตามลำดับก่อนหลัง ไม่แตกกิ่ง        | หน่วยที่ 3    |
| Selection (เลือก)  | แตกกิ่งเลือกทางใดทางหนึ่งตามเงื่อนไข    | หน่วยที่ 4    |
| Repetition (ทำซ้ำ) | วนกลับไปทำงานซ้ำจนกว่าเงื่อนไขจะเปลี่ยน | หน่วยที่ 5    |

2. หลักการทำงานของ Loop

Loop คือโครงสร้างที่ใช้สัญลักษณ์ซ้ำวนลมหดเดียวกับ Selection ในการตรวจสอบเงื่อนไข แต่มีจุดต่างสำคัญคือเมื่อทำงานตามคำสั่งภายใน Loop เสร็จแล้ว จะวนกลับไปตรวจสอบเงื่อนไขเดิมอีกครั้ง แทนที่จะเดินหน้าต่อไปแบบ Selection ทำเช่นนั้นซ้ำไปเรื่อย ๆ จนกว่าเงื่อนไขจะเปลี่ยนเป็นเท็จ จึงจะออกจาก Loop

ต่อไปนี้เป็นแผนภาพเปรียบเทียบโครงสร้าง Selection กับ Repetition เพื่อให้เห็นจุดต่างสำคัญคือ "เส้นทางวนกลับ"



### 3. องค์ประกอบสำคัญ 4 ส่วนของ Loop ทุกประเภท

ไม่ว่าจะใช้คำสั่ง Loop ชนิดใดในภาษา C# (while, do-while, for) ทุก Loop จะต้องมียุทธศาสตร์ประกอบครบทั้ง 4 ส่วนนี้เสมอ มิฉะนั้นจะเกิดปัญหา Loop ไม่รู้จบ (Infinite Loop)

| องค์ประกอบ                             | หน้าที่                                             | ตัวอย่าง              |
|----------------------------------------|-----------------------------------------------------|-----------------------|
| ① การกำหนดค่าเริ่มต้น (Initialization) | กำหนดค่าตั้งต้นของตัวแปรที่ใช้ควบคุม Loop           | int i = 1;            |
| ② เงื่อนไขการซ้ำ (Condition)           | ตรวจสอบว่าจะทำงานซ้ำต่อหรือหยุด                     | i <= 5                |
| ③ คำสั่งที่ต้องการทำซ้ำ (Body)         | ชุดคำสั่งที่ทำงานในแต่ละรอบ                         | Console.WriteLine(i); |
| ④ การเปลี่ยนค่าตัวแปรควบคุม (Update)   | เปลี่ยนค่าตัวแปรเพื่อให้ Loop ดำเนินไปสู่จุดสิ้นสุด | i++;                  |

**คำเตือนสำคัญ:** หากลืมองค์ประกอบข้อ ④ (การเปลี่ยนค่าตัวแปรควบคุม) เงื่อนไขจะเป็นจริงตลอดไปไม่มีวันเปลี่ยนเป็นเท็จ ทำให้เกิด Infinite Loop ซึ่งโปรแกรมจะค้างและไม่มีวันจบการทำงาน ผู้เรียนต้องตรวจสอบจุดนี้อย่างระมัดระวังทุกครั้ง

### 4. ประเภทของคำสั่ง Loop ในภาษา C#

| คำสั่ง   | ลักษณะเด่น                                                               | เหมาะกับสถานการณ์                    |
|----------|--------------------------------------------------------------------------|--------------------------------------|
| while    | ตรวจสอบเงื่อนไขก่อนเข้าทำงาน อาจไม่ทำงานเลยถ้าเงื่อนไขเป็นเท็จตั้งแต่แรก | ไม่ทราบจำนวนรอบที่แน่นอนล่วงหน้า     |
| do-while | ทำงานก่อน 1 รอบเสมอแล้วจึงตรวจสอบเงื่อนไขทีหลัง                          | ต้องการให้ทำงานอย่างน้อย 1 ครั้งเสมอ |
| for      | รวมองค์ประกอบทั้ง 4 ไว้ในบรรทัดเดียว กระชับที่สุด                        | ทราบจำนวนรอบที่แน่นอนล่วงหน้า        |

#### ตัวอย่างที่ 1: คำสั่ง while

```
using System;
class Program
{
    static void Main()
    {
        int i = 1;
        while (i <= 5)
        {
            Console.WriteLine("รอบที่ " + i);
            i++;
        }
    }
}
```



## ตัวอย่างที่ 2: คำสั่ง do-while

```
using System;
class Program
{
    static void Main()
    {
        int i = 1;
        do
        {
            Console.WriteLine("รอบที่ " + i);
            i++;
        } while (i <= 5);
    }
}
```

## ตัวอย่างที่ 3: คำสั่ง for

```
using System;
class Program
{
    static void Main()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine("รอบที่ " + i);
        }
    }
}
```

ผลลัพธ์ที่ได้เหมือนกันทั้ง 3 ตัวอย่าง

รอบที่ 1  
รอบที่ 2  
รอบที่ 3  
รอบที่ 4  
รอบที่ 5

**ข้อสังเกต:** แม้ทั้ง 3 ตัวอย่างจะให้ผลลัพธ์เหมือนกันทุกประการ แต่คำสั่ง for เขียนได้กระชับที่สุดเพราะรวมองค์ประกอบทั้ง 4 (Initialization, Condition, Body, Update) ไว้ในโครงสร้างเดียว จึงเป็นที่นิยมมากที่สุดเมื่อทราบจำนวนรอบที่แน่นอน

## 5. เปรียบเทียบพฤติกรรมพิเศษของ while กับ do-while

จุดต่างที่สำคัญที่สุดระหว่าง while และ do-while คือลำดับการตรวจสอบเงื่อนไข ลองพิจารณาตัวอย่างที่กำหนดเงื่อนไขเป็นเท็จตั้งแต่แรก

```
int i = 10;
// while: ตรวจสอบก่อน จึงไม่ทำงานเลยแม้แต่ครั้งเดียว
while (i <= 5)
{
    Console.WriteLine("รอบที่ " + i); // จะไม่ถูกแสดงผลเลย
}
// do-while: ทำงานก่อน 1 รอบเสมอ แล้วจึงตรวจสอบ
do
{
    Console.WriteLine("รอบที่ " + i); // จะถูกแสดงผล 1 ครั้ง แม้เงื่อนไขเป็นเท็จ
} while (i <= 5);
```

| คำสั่ง            | จำนวนรอบที่ทำงาน (เมื่อ i=10 ตั้งแต่แรก) |
|-------------------|------------------------------------------|
| while (i <= 5)    | 0 รอบ (ไม่ทำงานเลย)                      |
| do-while (i <= 5) | 1 รอบ (ทำงานก่อนตรวจสอบเสมอ)             |

## สรุปหลักการทำงานของ Loop

Loop คือโครงสร้างควบคุมที่สามที่ต่อยอดจาก Sequence และ Selection โดยอาศัยสัญลักษณ์ซ้ำหาลมตัดเดียวกัน ในการตรวจสอบเงื่อนไข แต่เพิ่มเส้นทางวนกลับเข้ามา ทำให้คำสั่งภายในทำงานซ้ำได้หลายรอบจนกว่าเงื่อนไขจะเป็นเท็จ ทุก Loop ต้องมีองค์ประกอบครบ 4 ส่วน (กำหนดค่าเริ่มต้น เงื่อนไข คำสั่งที่ทำซ้ำ และการเปลี่ยนค่าตัวแปรควบคุม) เพื่อป้องกัน ปัญหา Infinite Loop โดยภาษา C# มีคำสั่ง Loop ให้เลือกใช้ 3 รูปแบบคือ while, do-while และ for ซึ่งแต่ละแบบเหมาะกับสถานการณ์ที่แตกต่างกันไป

## 4.7 การเขียนโปรแกรมด้วยคำสั่ง Loop — ตัวอย่างประยุกต์ใช้จริง

จากหลักการทำงานของ Loop ที่ได้อธิบายไปก่อนหน้านี้ ต่อไปนี้เป็นตัวอย่างการประยุกต์ใช้คำสั่ง Loop ในการ แก้ปัญหาจริงหลากหลายรูปแบบ เรียงจากง่ายไปยาก เพื่อให้ผู้เรียนเห็นภาพการนำไปใช้งานที่กว้างขึ้น

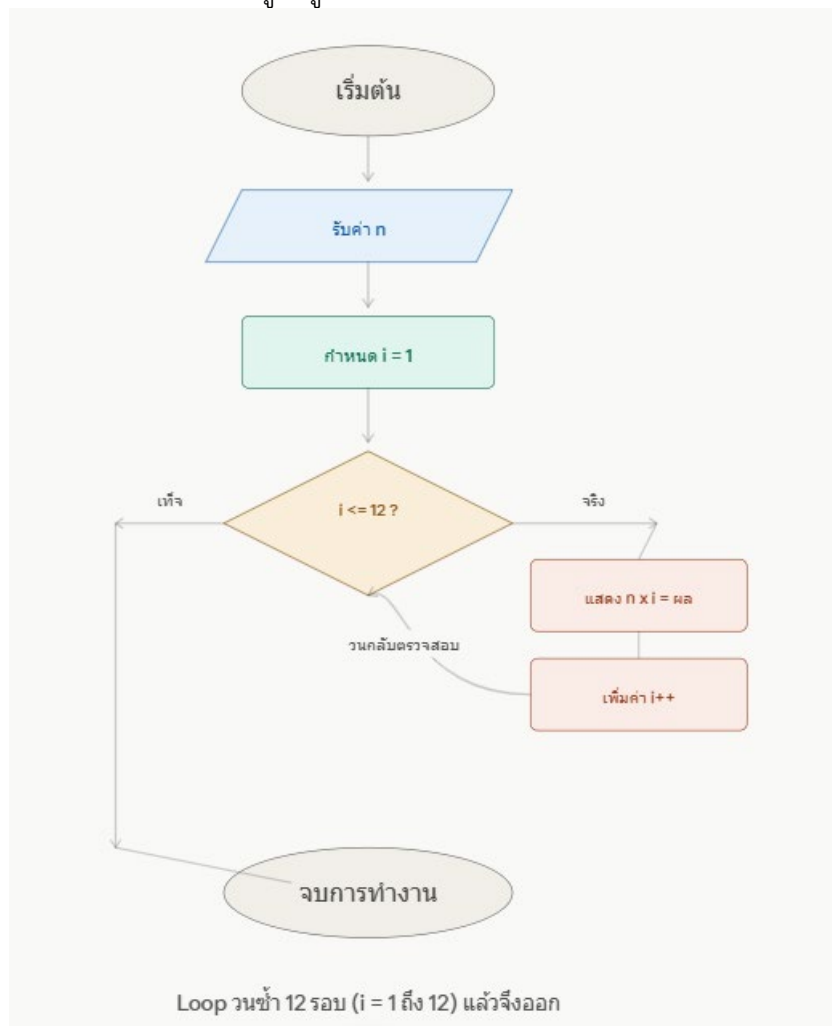
### ตัวอย่างที่ 1: โปรแกรมหาผลรวมของตัวเลข 1 ถึง N

โจทย์นี้แสดงการใช้ Loop ร่วมกับตัวแปรสะสมค่า (Accumulator) ซึ่งเป็นเทคนิคพื้นฐานที่สำคัญมาก

```
using System;
class Program
{
    static void Main()
    {
        Console.Write("กรอกค่า N: ");
        int n = int.Parse(Console.ReadLine());
        int sum = 0; // ตัวแปรสะสมผลรวม เริ่มต้นที่ 0 เสมอ
        for (int i = 1; i <= n; i++)
        {
            sum = sum + i; // สะสมค่า i เข้าไปในตัวแปร sum ทีละรอบ
        }
        Console.WriteLine("ผลรวมของเลข 1 ถึง " + n + " คือ " + sum);
    }
}
```

$$2 \times 12 = 24$$

ต่อไปนี้เป็นผังงานที่สอดคล้องกับโปรแกรมแสดงสูตรคูณข้างต้น เพื่อให้เห็นความสัมพันธ์ระหว่างผังงานกับโค้ด



### ตัวอย่างที่ 3: โปรแกรมตรวจสอบข้อมูลนำเข้าซ้ำจนกว่าจะถูกต้อง (Input Validation)

ตัวอย่างนี้แสดงการใช้ do-while ในสถานการณ์ที่เหมาะสมที่สุด คือต้องการให้รับค่าจากผู้ใช้อย่างน้อย 1 ครั้งเสมอ แล้ววนถามซ้ำหากค่าที่กรอกไม่ถูกต้อง

```
using System;
class Program
{
    static void Main()
    {
        int age;
        do
        {
            Console.WriteLine("กรอกอายุ (ต้องมากกว่า 0): ");
            age = int.Parse(Console.ReadLine());
            if (age <= 0)
            {
                Console.WriteLine("อายุไม่ถูกต้อง กรุณากรอกใหม่");
            }
        } while (age <= 0);
    }
}
```

```

        Console.WriteLine("อายุที่กรอกคือ " + age + " ปี ข้อมูลถูกต้อง");
    }
}

```

**ผลลัพธ์ตัวอย่าง** (กรอก -5 ก่อน แล้วกรอก 17)

กรอกอายุ (ต้องมากกว่า 0): -5

อายุไม่ถูกต้อง กรุณากรอกใหม่

กรอกอายุ (ต้องมากกว่า 0): 17

อายุที่กรอกคือ 17 ปี ข้อมูลถูกต้อง

**คำอธิบาย:** โปรแกรมนี้แสดงให้เห็นการผสมผสานระหว่าง Loop (do-while) กับ Selection (if) เข้าด้วยกัน ซึ่งเป็นรูปแบบที่พบได้บ่อยมากในการเขียนโปรแกรมจริง เพราะการตรวจสอบความถูกต้องของข้อมูลนำเข้า (Input Validation) มักต้องใช้ทั้งสองโครงสร้างร่วมกันเสมอ

**ตัวอย่างที่ 4: โปรแกรมนับจำนวนเลขคู่ในช่วงที่กำหนด**

ตัวอย่างนี้แสดงการผสมผสาน Loop เข้ากับ if เพื่อกรองเฉพาะข้อมูลที่ตรงเงื่อนไข

using System;

class Program

```

{
    static void Main()
    {
        Console.Write("กรอกค่า N: ");
        int n = int.Parse(Console.ReadLine());
        int countEven = 0;
        for (int i = 1; i <= n; i++)
        {
            if (i % 2 == 0)
            {
                countEven++;
            }
        }
        Console.WriteLine("จำนวนเลขคู่ตั้งแต่ 1 ถึง " + n + " มีทั้งหมด " + countEven + " ตัว");
    }
}

```

**ผลลัพธ์ตัวอย่าง** (กรอก N = 10)

กรอกค่า N: 10

จำนวนเลขคู่ตั้งแต่ 1 ถึง 10 มีทั้งหมด 5 ตัว

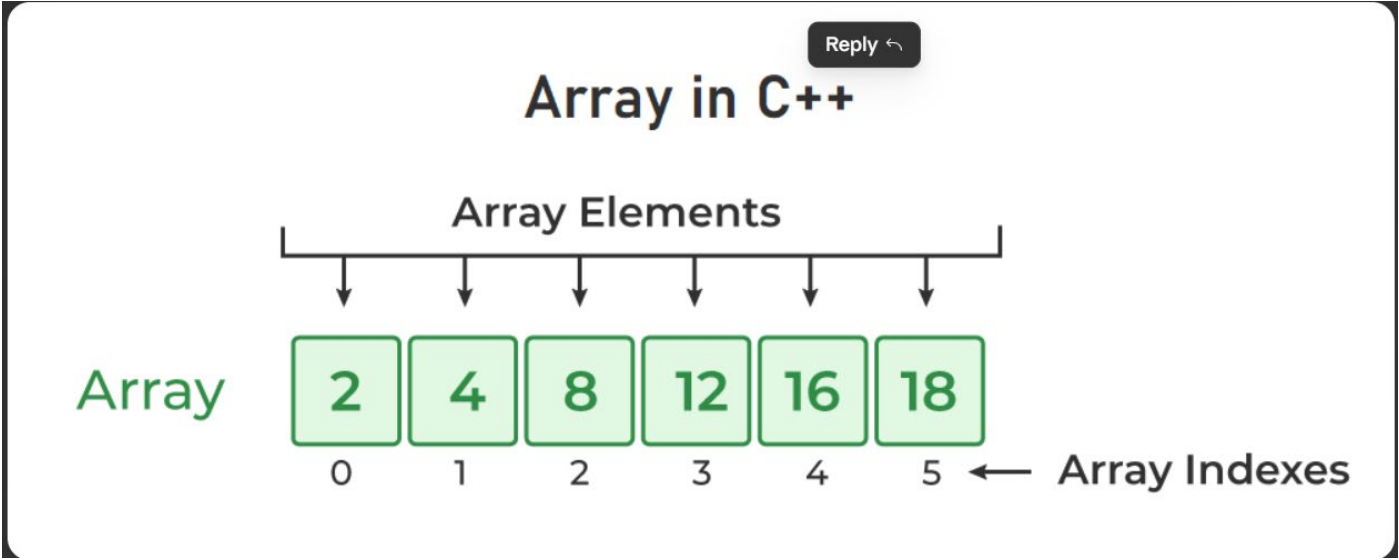


ตารางสรุปเทคนิคการเขียนโปรแกรมด้วย Loop ที่พบบ่อย

| เทคนิค                              | ลักษณะการใช้งาน                                  | ตัวอย่างที่แสดง             |
|-------------------------------------|--------------------------------------------------|-----------------------------|
| ตัวแปรสะสมค่า (Accumulator)         | กำหนดค่าเริ่มต้นก่อน Loop แล้วบวก/คูณสะสมทีละรอบ | ตัวอย่างที่ 1 (ผลรวม)       |
| การวนซ้ำตามจำนวนรอบที่แน่นอน        | ใช้ for เมื่อทราบจำนวนรอบล่วงหน้า                | ตัวอย่างที่ 2 (สูตรคูณ)     |
| การวนซ้ำจนกว่าเงื่อนไขจะถูกต้อง     | ใช้ do-while ร่วมกับ if ตรวจสอบข้อมูลนำเข้า      | ตัวอย่างที่ 3 (ตรวจสอบอายุ) |
| การนับจำนวนที่ตรงเงื่อนไข (Counter) | ใช้ for ร่วมกับ if เพื่อกรองและนับ               | ตัวอย่างที่ 4 (นับเลขคู่)   |

4.8 หลักการทำงานของคำสั่ง Array (อาร์เรย์)

ก่อนเริ่ม ขอตั้งข้อสังเกตเช่นเดียวกับหัวข้อ Loop ครั้น: **Array** เป็นเนื้อหาที่สอดคล้องกับไฟล์หน่วยที่ 10 ที่แนบมา ซึ่งตามโครงสร้างวิชามักจัดเป็นหน่วยแยกต่างหาก (เช่น หน่วยที่ 6) ถัดจากหน่วยควบคุมการทำซ้ำ เนื่องจาก Array มักถูกใช้งานร่วมกับ Loop เสมอ ผมจะสร้างเนื้อหาให้เห็นลักษณะเดียว



ที่มา <https://www.geeksforgeeks.org/cpp/cpp-arrays/>

1. เหตุผลที่ต้องมี Array

ลองจินตนาการสถานการณ์ที่ต้องเก็บคะแนนสอบของนักเรียน 30 คน หากใช้ตัวแปรแบบที่ผู้เรียนเคยศึกษามา (เช่น `int score1, score2, score3, ...`) จะต้องประกาศตัวแปรถึง 30 ตัว ซึ่งไม่สะดวกและไม่สามารถใช้ร่วมกับ Loop เพื่อประมวลผลข้อมูลจำนวนมากได้อย่างมีประสิทธิภาพ **Array (อาร์เรย์)** คือโครงสร้างข้อมูลที่ใช้เก็บข้อมูลชนิดเดียวกันหลายค่าไว้ภายใต้ชื่อตัวแปรเดียว โดยแต่ละค่าจะถูกอ้างอิงด้วยหมายเลขลำดับที่เรียกว่า ดัชนี (Index) ทำให้สามารถเก็บและประมวลผลข้อมูลจำนวนมากได้อย่างเป็นระบบ โดยเฉพาะเมื่อใช้ร่วมกับคำสั่ง Loop ที่ได้ศึกษามาก่อนหน้านี้

2. หลักการทำงานและโครงสร้างของ Array

Array เปรียบเสมือนตู้เก็บของที่มีช่องเรียงกันเป็นแถว โดยแต่ละช่องมีหมายเลขกำกับเริ่มต้นจาก 0 เสมอ (ไม่ใช่ 1) ซึ่งเป็นกฎสำคัญที่ผู้เรียนต้องจดจำ

int[] score = new int[5];

...

|          |          |          |          |          |
|----------|----------|----------|----------|----------|
| 60       | 75       | 88       | 92       | 70       |
| score[0] | score[1] | score[2] | score[3] | score[4] |
| ดัชนี 0  | ดัชนี 1  | ดัชนี 2  | ดัชนี 3  | ดัชนี 4  |

Array ขนาด 5 ช่อง มีดัชนีตั้งแต่ 0 ถึง 4 เสมอ (ไม่ใช่ 1 ถึง 5)

### 3. การประกาศและกำหนดค่า Array ในภาษา C#

#### รูปแบบการประกาศ

ชนิดข้อมูล[ ] ชื่อตัวแปร = new ชนิดข้อมูล[ขนาด];

int[] score = new int[5]; // ประกาศ array เก็บจำนวนเต็ม ขนาด 5 ช่อง (ดัชนี 0-4)

การกำหนดค่าให้แต่ละช่อง

csharp

score[0] = 60;

score[1] = 75;

score[2] = 88;

score[3] = 92;

score[4] = 70;

การประกาศพร้อมกำหนดค่าเริ่มต้นในบรรทัดเดียว (นิยมใช้มากกว่า)

int[] score = { 60, 75, 88, 92, 70 };

### 4. การเข้าถึงข้อมูลใน Array

การเข้าถึงค่าในแต่ละช่องทำได้โดยระบุชื่อตัวแปรตามด้วยหมายเลขดัชนีในเครื่องหมาย [ ]

using System;

class Program

{

static void Main()

{

int[] score = { 60, 75, 88, 92, 70 };

Console.WriteLine("คะแนนช่องแรก (ดัชนี 0): " + score[0]);

Console.WriteLine("คะแนนช่องสุดท้าย (ดัชนี 4): " + score[4]);

score[2] = 95; // เปลี่ยนค่าในดัชนีที่ 2 จาก 88 เป็น 95

Console.WriteLine("คะแนนหลังแก้ไข (ดัชนี 2): " + score[2]);

}

}

## ผลลัพธ์

คะแนนช่องแรก (ดัชนี 0): 60

คะแนนช่องสุดท้าย (ดัชนี 4): 70

คะแนนหลังแก้ไข (ดัชนี 2): 95

**ข้อควรระวังสำคัญ:** หากอ้างอิงดัชนีที่เกินขอบเขตของ Array เช่น score[5] (ทั้งที่ Array มีเพียง 5 ช่อง คือดัชนี 0-4) โปรแกรมจะเกิดข้อผิดพลาด `IndexOutOfRangeException` ทันทีขณะรันโปรแกรม ผู้เรียนต้องตรวจสอบขอบเขตของ Array อย่างระมัดระวังทุกครั้ง

### 5. การใช้ Loop ร่วมกับ Array (จุดสำคัญที่สุด)

พลังที่แท้จริงของ Array จะปรากฏชัดเมื่อใช้ร่วมกับคำสั่ง Loop เพราะสามารถประมวลผลข้อมูลจำนวนมากได้ด้วยโค้ดเพียงไม่กี่บรรทัด แทนที่จะเขียนอ้างอิงทีละดัชนีแบบตัวอย่างข้างต้น

**ตัวอย่าง: แสดงค่าทั้งหมดใน Array ด้วย for loop**

```
using System;
```

```
class Program
```

```
{  
    static void Main()  
    {  
        int[] score = { 60, 75, 88, 92, 70 };  
  
        for (int i = 0; i < score.Length; i++)  
        {  
            Console.WriteLine("คะแนนคนที่ " + (i + 1) + ": " + score[i]);  
        }  
    }  
}
```

## ผลลัพธ์

คะแนนคนที่ 1: 60

คะแนนคนที่ 2: 75

คะแนนคนที่ 3: 88

คะแนนคนที่ 4: 92

คะแนนคนที่ 5: 70

**คำอธิบายสำคัญ:** `score.Length` คือคุณสมบัติที่คืนค่าจำนวนช่องทั้งหมดของ Array (ในที่นี้คือ 5) การใช้ `i < score.Length` แทนการเขียนตัวเลข 5 ตรง ๆ ทำให้โค้ดยืดหยุ่น หาก Array มีขนาดเปลี่ยนไปในอนาคต โค้ดส่วนนี้ก็ยังทำงานถูกต้องโดยไม่ต้องแก้ไข

**ตัวอย่าง: หาผลรวมและค่าเฉลี่ยของคะแนนใน Array**

```
using System;
```

```
class Program
```

```
{  
    static void Main()  
    {  
        int[] score = { 60, 75, 88, 92, 70 };  
        int sum = 0;
```

```
for (int i = 0; i < score.Length; i++)
{
    sum = sum + score[i];
}
double average = (double)sum / score.Length;
Console.WriteLine("ผลรวมคะแนนทั้งหมด: " + sum);
Console.WriteLine("คะแนนเฉลี่ย: " + average);
}
```

ผลลัพธ์

ผลรวมคะแนนทั้งหมด: 385

คะแนนเฉลี่ย: 77

**เทคนิคที่ควรสังเกต:** โค้ดนี้ผสมผสาน 3 แนวคิดที่เคยเรียนมาทั้งหมดเข้าด้วยกัน คือ **Array** (เก็บข้อมูล) **Loop** (วนอ่านค่าทีละช่อง) และ **ตัวแปรสะสมค่า** (คำนวณผลรวม) ซึ่งเป็นรูปแบบการเขียนโปรแกรมที่พบบ่อยที่สุดในการประมวลผลข้อมูลชุดใหญ่

6. ตารางสรุปคำสั่งสำคัญเกี่ยวกับ Array

| คำสั่ง/ไวยากรณ์                    | ความหมาย                                                |
|------------------------------------|---------------------------------------------------------|
| int[] a = new int[5];              | ประกาศ array ขนาด 5 ช่อง (ดัชนี 0-4)                    |
| int[] a = {1,2,3};                 | ประกาศพร้อมกำหนดค่าเริ่มต้นทันที                        |
| a[0]                               | อ้างอิงค่าในช่องแรก (ดัชนี 0)                           |
| a[a.Length - 1]                    | อ้างอิงค่าในช่องสุดท้ายเสมอ ไม่ว่า Array จะมีขนาดเท่าใด |
| a.Length                           | คืนค่าจำนวนช่องทั้งหมดของ array                         |
| for (int i = 0; i < a.Length; i++) | รูปแบบมาตรฐานสำหรับวนอ่านค่าทุกช่องใน array             |

4.9 การเขียนโปรแกรมด้วยคำสั่ง Array — ตัวอย่างประยุกต์ใช้จริง

ต่อจากหลักการพื้นฐานที่อธิบายไปก่อนหน้านี้ ต่อไปนี้เป็นตัวอย่างโปรแกรมประยุกต์ใช้ Array ในสถานการณ์ที่พบบ่อย ซึ่งล้วนอาศัยการผสมผสาน Array เข้ากับ Loop และ if ที่ได้ศึกษามาแล้วทั้งหมด

ตัวอย่างที่ 1: โปรแกรมหาค่ามากที่สุด (Maximum) ในชุดข้อมูล

โจทย์นี้เป็นเทคนิคพื้นฐานสำคัญที่เรียกว่า "ตัวแปรเก็บค่าที่ดีที่สุด" (Best-so-far Variable) ซึ่งใช้หลักการสมมติค่าแรกเป็นค่ามากที่สุดไว้ก่อน แล้วไล่เปรียบเทียบกับค่าที่เหลือทีละตัว

```
using System;
class Program
{
    static void Main()
    {
        int[] score = { 60, 92, 75, 88, 70 };
        int max = score[0]; // สมมติค่าแรกเป็นค่ามากที่สุดไว้ก่อน
        for (int i = 1; i < score.Length; i++)
        {
            if (score[i] > max)
            {
```

```

        max = score[i];    // พบค่าที่มากกว่า จึงปรับปรุงค่า max
    }
}
Console.WriteLine("คะแนนสูงสุดคือ: " + max);
}
}

```

Trace Table แสดงการทำงานที่ละรอบ

| รอบที่ (i) | ค่า score[i] | เปรียบเทียบ score[i] > max | ค่า max หลังเปรียบเทียบ      |
|------------|--------------|----------------------------|------------------------------|
| เริ่มต้น   | -            | -                          | 60 (ค่าเริ่มต้นจาก score[0]) |
| 1          | 92           | 92 > 60 จริง               | 92                           |
| 2          | 75           | 75 > 92 เท็จ               | 92 (ไม่เปลี่ยน)              |
| 3          | 88           | 88 > 92 เท็จ               | 92 (ไม่เปลี่ยน)              |
| 4          | 70           | 70 > 92 เท็จ               | 92 (ไม่เปลี่ยน)              |

ผลลัพธ์

คะแนนสูงสุดคือ: 92

**ข้อสังเกต:** การหาค่าน้อยที่สุด (Minimum) ใช้หลักการเดียวกันทุกประการ เพียงเปลี่ยนเงื่อนไขจาก `score[i] > max` เป็น `score[i] < min` เท่านั้น

### ตัวอย่างที่ 2: โปรแกรมค้นหาข้อมูลใน Array (Linear Search)

การค้นหาข้อมูลใน Array ทำได้โดยการไล่เปรียบเทียบทีละช่องตั้งแต่ต้นจนพบ ซึ่งเป็นวิธีค้นหาพื้นฐานที่สุดที่เรียกว่า Linear Search

```

using System;
class Program

```

```

{
    static void Main()
    {
        string[] names = { "สมชาย", "สมหญิง", "วิภา", "ประยุทธ์", "มานี" };
        Console.Write("กรอกชื่อที่ต้องการค้นหา: ");
        string searchName = Console.ReadLine();
        bool found = false;
        for (int i = 0; i < names.Length; i++)
        {
            if (names[i] == searchName)
            {
                Console.WriteLine("พบชื่อ " + searchName + " อยู่ที่ลำดับที่ " + (i + 1));
                found = true;
                break;        // พบแล้วออกจาก Loop ทันที ไม่ต้องค้นต่อ
            }
        }
        if (!found)
        {
            Console.WriteLine("ไม่พบชื่อ " + searchName + " ในรายชื่อ");
        }
    }
}

```

```
}  
}
```

**ผลลัพธ์ตัวอย่าง** (ค้นหา "วิชา")

กรอกชื่อที่ต้องการค้นหา: วิชา

พบชื่อ วิชา อยู่ลำดับที่ 3

**เทคนิคสำคัญ:** คำสั่ง break ในที่นี้ทำหน้าที่หยุดการค้นหาทันทีที่พบข้อมูลที่ต้องการ ไม่จำเป็นต้องไล่ตรวจสอบช่องที่เหลือต่อไป ช่วยให้โปรแกรมทำงานมีประสิทธิภาพมากขึ้น โดยเฉพาะเมื่อ Array มีขนาดใหญ่มาก

**ตัวอย่างที่ 3:** โปรแกรมนับจำนวนข้อมูลที่ตรงเงื่อนไขใน Array

```
using System;  
class Program  
{  
    static void Main()  
    {  
        int[] score = { 45, 78, 55, 90, 38, 62, 71 };  
        int countPass = 0;  
  
        for (int i = 0; i < score.Length; i++)  
        {  
            if (score[i] >= 50)  
            {  
                countPass++;  
            }  
        }  
        Console.WriteLine("จำนวนนักเรียนที่สอบผ่าน: " + countPass + " คน จากทั้งหมด " + score.Length + " คน");  
    }  
}
```

**ผลลัพธ์**

จำนวนนักเรียนที่สอบผ่าน: 5 คน จากทั้งหมด 7 คน

**ตัวอย่างที่ 4:** โปรแกรมกลับลำดับข้อมูลใน Array (Reverse)

ตัวอย่างนี้แสดงเทคนิคการอ้างอิงดัชนีแบบย้อนกลับ ซึ่งอาศัยความเข้าใจเรื่อง Length และดัชนีสุดท้าย (Length - 1)

```
using System;  
class Program  
{  
    static void Main()  
    {  
        int[] number = { 10, 20, 30, 40, 50 };  
        Console.Write("ลำดับเดิม: ");  
        for (int i = 0; i < number.Length; i++)  
        {  
            Console.Write(number[i] + " ");  
        }  
    }  
}
```



```
Console.WriteLine();
Console.Write("ลำดับย้อนกลับ: ");
for (int i = number.Length - 1; i >= 0; i--)
{
    Console.Write(number[i] + " ");
}
}
```

ผลลัพธ์

ลำดับเดิม: 10 20 30 40 50  
ลำดับย้อนกลับ: 50 40 30 20 10

**คำอธิบาย:** การวนซ้ำแบบย้อนกลับใช้หลักการเดียวกับ for ปกติ เพียงแต่กำหนดค่าเริ่มต้น i เป็น number.Length - 1 (ดัชนีสุดท้าย) เงื่อนไขเป็น i >= 0 และการเปลี่ยนค่าเป็น i-- (ลดค่าแทนที่จะเพิ่ม)

ตารางสรุปรูปแบบการเขียนโปรแกรม Array ที่พบบ่อย

| รูปแบบปัญหา               | เทคนิคที่ใช้                           | ตัวอย่างที่แสดง |
|---------------------------|----------------------------------------|-----------------|
| หาค่ามากที่สุด/น้อยที่สุด | ตัวแปรเก็บค่าที่ดีที่สุด (Best-so-far) | ตัวอย่างที่ 1   |
| ค้นหาข้อมูล               | Linear Search พร้อม break เมื่อพบ      | ตัวอย่างที่ 2   |
| นับข้อมูลที่ตรงเงื่อนไข   | Loop ร่วมกับ if และตัวแปรนับ (Counter) | ตัวอย่างที่ 3   |
| กลับลำดับข้อมูล           | Loop ย้อนกลับโดยเริ่มจาก Length-1      | ตัวอย่างที่ 4   |

4.10 การประยุกต์ใช้ตัวดำเนินการเชิงตรรกะร่วมกับเงื่อนไข

4.10.1 ทบทวนและเหตุผลที่ต้องใช้ตัวดำเนินการเชิงตรรกะ

ในสถานการณ์จริง เงื่อนไขมักมีความซับซ้อนมากกว่าการเปรียบเทียบเพียงค่าเดียว เช่น การตรวจสอบสิทธิ์ที่ต้องพิจารณาทั้งอายุและเกรดเฉลี่ยพร้อมกัน หรือการตรวจสอบว่าตัวเลขอยู่นอกช่วงที่กำหนด ซึ่งการเขียน if เพียงเงื่อนไขเดียวไม่สามารถครอบคลุมได้ ภาษา C# จึงมีตัวดำเนินการเชิงตรรกะ (Logical Operator) ที่ได้ศึกษาในหน่วยที่ 3 มาประยุกต์ใช้ร่วมกับคำสั่งตรวจสอบเงื่อนไข ได้แก่ && (AND), || (OR) และ ! (NOT)

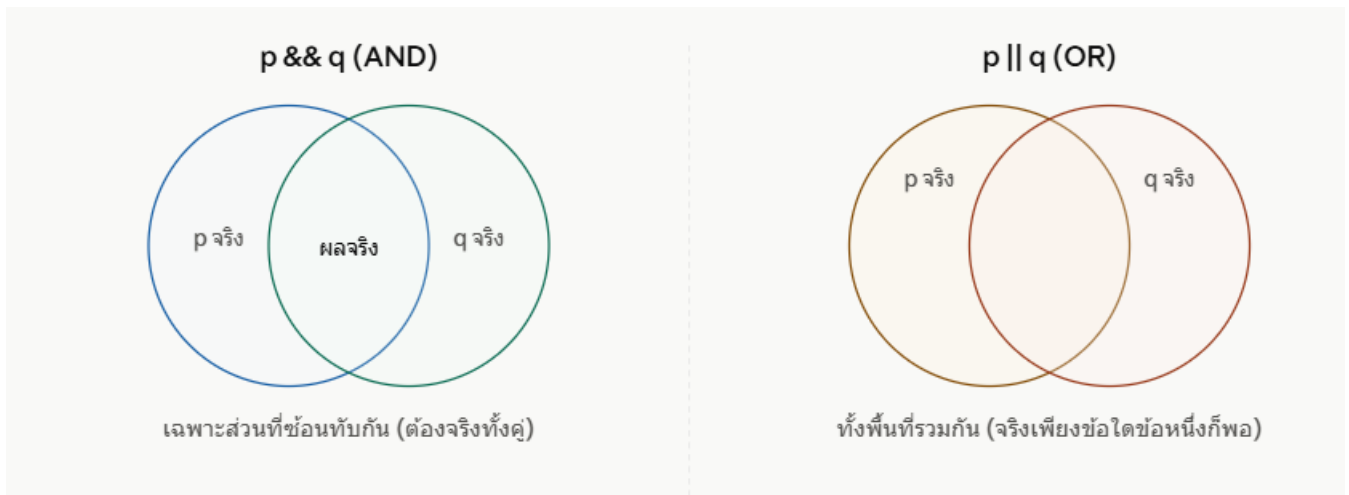
4.10.2 ตารางค่าความจริง (Truth Table) ของตัวดำเนินการเชิงตรรกะ

| p    | q    | p && q (AND) | p    q (OR) | !p (NOT) |
|------|------|--------------|-------------|----------|
| จริง | จริง | จริง         | จริง        | เท็จ     |
| จริง | เท็จ | เท็จ         | จริง        | เท็จ     |
| เท็จ | จริง | เท็จ         | จริง        | จริง     |
| เท็จ | เท็จ | เท็จ         | เท็จ        | จริง     |

หลักการจำง่าย

- && (AND) — ผลลัพธ์เป็นจริง ก็ต่อเมื่อทั้งสองเงื่อนไขเป็นจริงพร้อมกัน เท่านั้น
- || (OR) — ผลลัพธ์เป็นจริง เมื่อเงื่อนไขใดเงื่อนไขหนึ่งเป็นจริง ก็เพียงพอแล้ว
- ! (NOT) — กลับค่าความจริงให้เป็นตรงข้ามเสมอ

ต่อไปนี้เป็นแผนภาพเปรียบเทียบพฤติกรรมของ && และ || เพื่อให้เห็นความแตกต่างชัดเจนยิ่งขึ้น



#### 4.10.3 ตัวอย่างที่ 1: การใช้ && (AND) — ตรวจสอบสิทธิ์การสมัครเรียน

```
using System;
class Program
{
    static void Main()
    {
        Console.Write("กรอกอายุ: ");
        int age = int.Parse(Console.ReadLine());
        Console.Write("กรอกเกรดเฉลี่ย: ");
        double gpa = double.Parse(Console.ReadLine());
        if (age >= 15 && gpa >= 2.00)
        {
            Console.WriteLine("มีสิทธิ์สมัครเรียน");
        }
        else
        {
            Console.WriteLine("ไม่มีสิทธิ์สมัครเรียน");
        }
    }
}
```

##### ตารางทดสอบผลลัพธ์ในหลายกรณี

| อายุ | เกรดเฉลี่ย | age>=15 | gpa>=2.00 | ผลลัพธ์รวม (&&) | สิทธิ์สมัครเรียน |
|------|------------|---------|-----------|-----------------|------------------|
| 17   | 2.50       | จริง    | จริง      | จริง            | ✅ มีสิทธิ์       |
| 17   | 1.80       | จริง    | เท็จ      | เท็จ            | ❌ ไม่มีสิทธิ์    |
| 13   | 3.00       | เท็จ    | จริง      | เท็จ            | ❌ ไม่มีสิทธิ์    |
| 13   | 1.50       | เท็จ    | เท็จ      | เท็จ            | ❌ ไม่มีสิทธิ์    |

**ข้อสังเกต:** จากตารางจะเห็นว่า มีเพียงกรณีเดียวเท่านั้นที่ได้สิทธิ์สมัครเรียน คือกรณีที่ทั้งสองเงื่อนไขเป็นจริงพร้อมกัน สอดคล้องกับหลักการของ && ที่ต้องการให้ทุกเงื่อนไขเป็นจริงทั้งหมด

#### 4.10.4 ตัวอย่างที่ 2: การใช้ || (OR) — ตรวจสอบวันหยุดสุดสัปดาห์

using System;

class Program

```
{
    static void Main()
    {
        Console.Write("กรอกชื่อวัน (เช่น เสาร์, อาทิตย์, จันทร์): ");
        string day = Console.ReadLine();

        if (day == "เสาร์" || day == "อาทิตย์")
        {
            Console.WriteLine("วันนี้เป็นวันหยุดสุดสัปดาห์");
        }
        else
        {
            Console.WriteLine("วันนี้เป็นวันทำงาน");
        }
    }
}
```

**ผลลัพธ์ตัวอย่าง** (กรอก "เสาร์")

กรอกชื่อวัน (เช่น เสาร์, อาทิตย์, จันทร์): เสาร์

วันนี้เป็นวันหยุดสุดสัปดาห์

**คำอธิบาย:** เงื่อนไข `day == "เสาร์" || day == "อาทิตย์"` เป็นจริงเมื่อค่าตัวแปร `day` ตรงกับค่าใดค่าหนึ่งใน 2 คำนี้ก็เพียงพอแล้ว ไม่จำเป็นต้องตรงกับทั้งสองคำพร้อมกัน (ซึ่งเป็นไปไม่ได้อยู่แล้ว เพราะตัวแปรหนึ่งตัวเก็บค่าได้เพียงค่าเดียว)

#### 4.10.5 ตัวอย่างที่ 3: การใช้ ! (NOT) — ตรวจสอบสถานะตรงข้าม

using System;

class Program

```
{
    static void Main()
    {
        bool isGraduated = false;
        if (!isGraduated)
        {
            Console.WriteLine("ยังไม่จบการศึกษา ต้องลงทะเบียนเรียนต่อ");
        }
        else
        {
            Console.WriteLine("จบการศึกษาแล้ว");
        }
    }
}
```

## ผลลัพธ์

ยังไม่จบการศึกษา ต้องลงทะเบียนเรียนต่อ

**คำอธิบาย:** !isGraduated คือการกลับค่าตรงข้ามของตัวแปร isGraduated เมื่อ isGraduated มีค่าเป็น false ผลลัพธ์ของ !isGraduated จะเป็น true จึงเข้าทำงานในบล็อก if

### 4.10.6 ตัวอย่างที่ 4: การผสมผสานตัวดำเนินการหลายตัวในเงื่อนไขเดียว

โจทย์ที่ซับซ้อนขึ้น: โปรแกรมตรวจสอบว่าตัวเลขอยู่ในช่วงที่กำหนด (ระหว่าง 1 ถึง 100) หรือไม่

```
using System;
class Program
{
    static void Main()
    {
        Console.Write("กรอกตัวเลข: ");
        int number = int.Parse(Console.ReadLine());
        if (number >= 1 && number <= 100)
        {
            Console.WriteLine(number + " อยู่ในช่วง 1 ถึง 100");
        }
        else
        {
            Console.WriteLine(number + " ไม่อยู่ในช่วง 1 ถึง 100");
        }
    }
}
```

**ข้อสังเกตสำคัญ:** การตรวจสอบ "ช่วงค่า" ในภาษา C# ไม่สามารถเขียนแบบคณิตศาสตร์  $1 \leq \text{number} \leq 100$  ได้โดยตรง (ต่างจากที่เขียนบนกระดาษ) จำเป็นต้องแยกเป็น 2 เงื่อนไขแล้วเชื่อมด้วย && เสมอ คือ  $\text{number} \geq 1 \ \&\& \ \text{number} \leq 100$

**ตัวอย่างที่ซับซ้อนยิ่งขึ้น:** ผสาน && และ || ในเงื่อนไขเดียวกัน

```
int age = 16;
string status = "นักเรียน";
if ((age < 18) && (status == "นักเรียน" || status == "นักศึกษา"))
{
    Console.WriteLine("ได้รับส่วนลดพิเศษ 50%");
}
```

**คำอธิบาย:** การใช้วงเล็บ ( ) ช่วยกำหนดลำดับการประเมินผลให้ชัดเจน ในที่นี้เงื่อนไขจะเป็นจริงเมื่อ **อายุน้อยกว่า 18 ปี** และ **(เป็นนักเรียน หรือ นักศึกษา)** พร้อมกัน การใช้วงเล็บช่วยให้อ่านเงื่อนไขที่ซับซ้อนได้ง่ายขึ้นและป้องกันความผิดพลาดจากลำดับการประเมินผล

#### 4.10.7 ตารางสรุปการใช้ตัวดำเนินการเชิงตรรกะในสถานการณ์ต่าง ๆ

| สถานการณ์                           | ตัวดำเนินการที่ใช้      | ตัวอย่างเงื่อนไข                              |
|-------------------------------------|-------------------------|-----------------------------------------------|
| ต้องเป็นจริงทั้งสองเงื่อนไขพร้อมกัน | &&                      | if (age >= 15 && gpa >= 2.00)                 |
| เป็นจริงเงื่อนไขใดเงื่อนไขหนึ่งก็พอ |                         | if (day == "เสาร์"    day == "อาทิตย์")       |
| ต้องการกลับค่าความจริงตรงข้าม       | !                       | if (!isGraduated)                             |
| ตรวจสอบว่าค่าอยู่ในช่วงที่กำหนด     | && (สองเงื่อนไขร่วมกัน) | if (number >= 1 && number <= 100)             |
| เงื่อนไขซับซ้อนหลายชั้น             | && ผสม    พร้อมวงเล็บ   | if ((age<18) && (status=="A"    status=="B")) |

### สรุปท้ายบทเรียน หน่วยที่ 4: คำสั่งการตรวจสอบและควบคุม

#### 4.1 หลักการของคำสั่งตรวจสอบเงื่อนไข

หน่วยนี้ต่อยอดจากการเขียนโปรแกรมแบบลำดับในหน่วยที่ 3 สู่การเขียนโปรแกรมแบบเลือก (Selection) ซึ่งทำให้โปรแกรมสามารถ "ตัดสินใจ" เลือกเส้นทางการทำงานตามผลการประเมินเงื่อนไข (true/false) โดยมีสัญลักษณ์ผังงานใหม่ที่สำคัญคือ **รูปสี่เหลี่ยมข้าวหลามตัด (Decision Symbol)** ซึ่งมีลักษณะพิเศษคือมีทางออก 2 ทางเสมอ ทำให้ผังงานเริ่มมีการแตกกิ่งก้าน (Branching) ต่างจากผังงานเส้นตรงในหน่วยที่ 3

#### 4.2 คำสั่ง if และ if-else

คำสั่งพื้นฐานที่สุดของการตรวจสอบเงื่อนไข แบ่งเป็น 2 รูปแบบ คือ **if แบบทางเดียว** (ทำงานเฉพาะเมื่อเงื่อนไขเป็นจริง) และ **if-else** (เลือกทำงานอย่างใดอย่างหนึ่งจาก 2 ทางเลือก) เป็นรากฐานสำคัญที่ทุกคำสั่งตรวจสอบเงื่อนไขในหัวข้อถัดไปต่อยอดมาจากโครงสร้างนี้

#### 4.3 คำสั่ง if-else if

เมื่อสถานการณ์มีมากกว่า 2 ทางเลือก ใช้ **if-else if** ตรวจสอบเงื่อนไขต่อเนื่องกันจากบนลงล่าง โดยหยุดที่เงื่อนไขแรกที่เป็นจริงและไม่ตรวจสอบเงื่อนไขที่เหลือ ข้อควรระวังสำคัญที่สุดคือ **การเรียงลำดับเงื่อนไข** ต้องสอดคล้องกับทิศทางของตัวดำเนินการเปรียบเทียบเสมอ (เช่น เรียงจากมากไปน้อยเมื่อใช้ >=) มิฉะนั้นผลลัพธ์จะผิดพลาด ดังตัวอย่างการตัดเกรด A-B-C-D-F

#### 4.4 คำสั่ง switch-case

ทางเลือกสำหรับเปรียบเทียบค่าตัวแปรตัวเดียวกับค่าคงที่ที่แน่นอนหลายค่า (Equality) ทำให้โค้ดอ่านง่ายกว่า if-else if ในบางสถานการณ์ ประกอบด้วยคำสั่งสำคัญ switch, case, break และ default โดยข้อควรระวังที่สำคัญที่สุดคือ **ต้องมี break ทุก case** เพื่อป้องกันปัญหา Fall-through ที่โปรแกรมไหลต่อไปทำงานใน case ถัดไปโดยไม่หยุด

#### 4.5 การประยุกต์ใช้ตัวดำเนินการเชิงตรรกะร่วมกับเงื่อนไข

เมื่อเงื่อนไขมีความซับซ้อนเกินกว่าการเปรียบเทียบค่าเดียว ใช้ตัวดำเนินการเชิงตรรกะ && (ต้องจริงทั้งคู่), || (จริงข้อใดข้อหนึ่งพอ) และ ! (กลับค่าตรงข้าม) มาประกอบร่วมกับ if ซึ่งเป็นเทคนิคสำคัญสำหรับการตรวจสอบเงื่อนไขที่ใกล้เคียงสถานการณ์จริงมากขึ้น เช่น การตรวจสอบช่วงค่าหรือการตรวจสอบหลายปัจจัยพร้อมกัน

#### ตารางสรุปคำสั่งทั้งหมดในหน่วยที่ 4

| คำสั่ง/แนวคิด | ใช้เมื่อใด                      | ตัวอย่างเงื่อนไข                           |
|---------------|---------------------------------|--------------------------------------------|
| if            | ตรวจสอบเงื่อนไขทางเดียว         | if (score >= 50)                           |
| if-else       | เลือกจาก 2 ทางเลือก             | if (score >= 50) ... else ...              |
| if-else if    | ตรวจสอบหลายเงื่อนไขแบบช่วงค่า   | if (score>=80) ... else if (score>=70) ... |
| switch-case   | เปรียบเทียบค่าที่แน่นอนหลายค่า  | switch(grade) { case "A": ... }            |
| &&            | ต้องจริงทุกเงื่อนไข             | if (age>=15 && gpa>=2.00)                  |
|               | จริงเงื่อนไขใดเงื่อนไขหนึ่งก็พอ | if (day=="เสาร์"    day=="อาทิตย์")        |
| !             | กลับค่าความจริง                 | if (!isGraduated)                          |